Experiment 9: Implementing Monitoring using Prometheus / Grafana / Nagios

Experiment No.	9	Mapped CO	CO5
Subject	Agile Software Development and DevOps Lab	Course Code	2175119

Aim

To monitor system and application metrics by collecting them with Prometheus, visualizing them in Grafana and configuring an alert.

Tools / Software

Prometheus; Grafana; Node Exporter or an instrumented application; Docker optional; Web browser; Terminal / Command Prompt.

Theory

Monitoring provides visibility into the health and performance of systems through metrics, dashboards and alerts. Prometheus is a pull-based monitoring system that scrapes numeric time-series metrics from instrumented targets at fixed intervals and stores them with labels. These metrics can be queried using PromQL.

Node Exporter exposes host-level metrics such as CPU, memory, disk and network usage. Grafana connects to Prometheus as a data source and presents metrics in interactive dashboards. Alerting rules can be configured in Prometheus, Alertmanager or Grafana to notify users when selected metric thresholds are crossed. Nagios can also be used as a check-based monitoring tool for host and service status.

Procedure

1. Run Node Exporter on the host machine to expose system metrics on port 9100.

2. Configure prometheus.yml with a scrape job targeting the exporter and start Prometheus.

3. Verify that the target status is UP in the Prometheus web interface and execute a PromQL query.

4. Start Grafana, add Prometheus as a data source and import or create a monitoring dashboard.

5. Create an alert rule such as high CPU usage or high memory usage and confirm that it triggers under load.

6. Record screenshots or copied outputs in the observation boxes provided below.

Sample Code / Commands

prometheus.yml scrape configuration

```
global:
  scrape_interval: 15s
```

```
scrape_configs:
  - job_name: 'node'
    static_configs:
      - targets: ['localhost:9100']
```

Sample PromQL query for CPU utilization

```
100 - (avg by (instance)
  (rate(node_cpu_seconds_total{mode="idle"}[5m])) * 100)
```

Useful execution / verification commands

```
# Start Node Exporter and Prometheus as per installation method
# Verify targets in browser: http://localhost:9090/targets
# Open Grafana in browser: http://localhost:3000
# Add Prometheus data source URL: http://localhost:9090
```

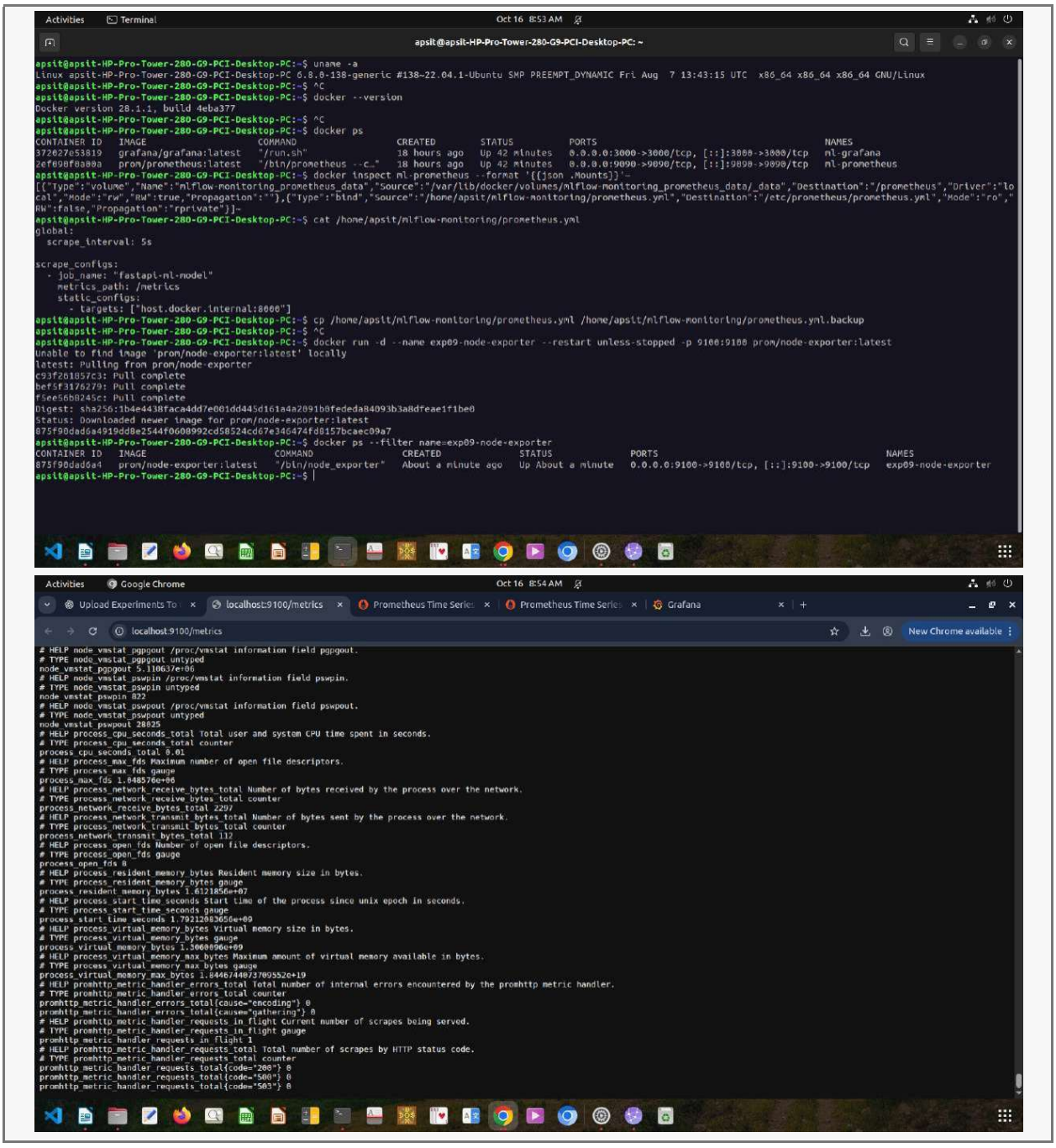
Observation / Experiment Output

Instructions to students: Paste screenshots or copied outputs in the boxes below after completing Node Exporter setup, Prometheus configuration, Grafana dashboard creation and alert configuration. Keep screenshots clear and readable.

Name	Roll No.	Batch	Date
Vighnesh Pingale	24107108	B2	16-10-2026

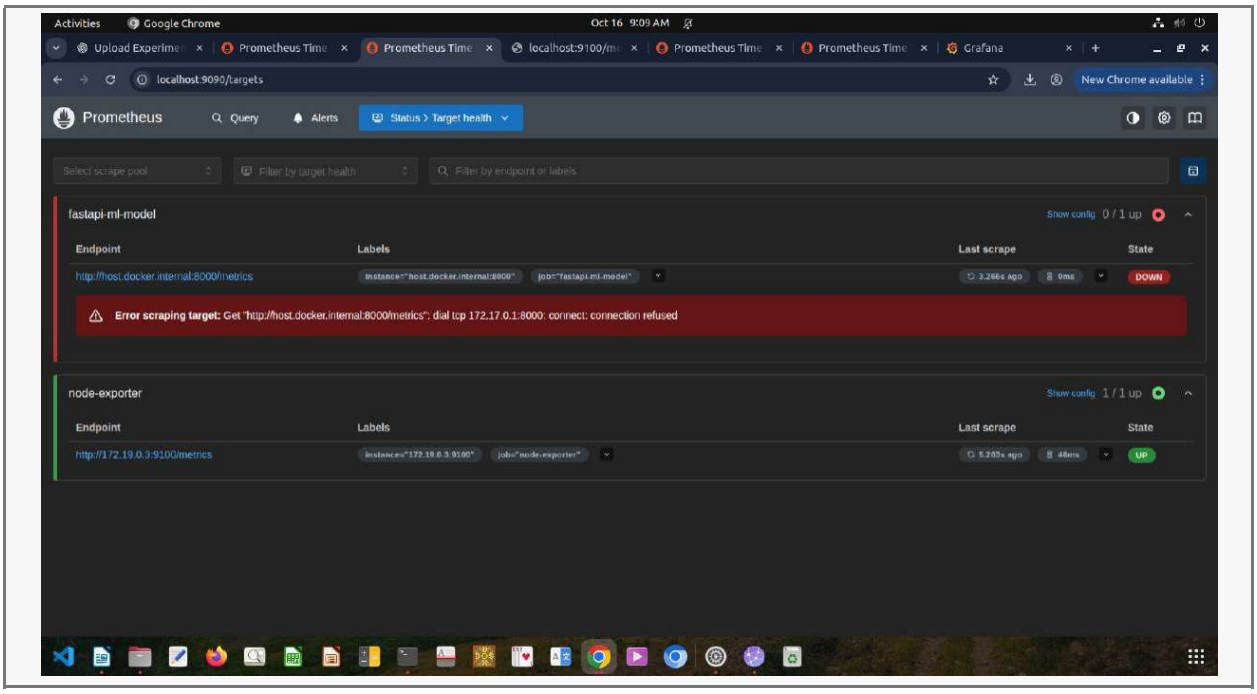
1. Node Exporter / Metrics Endpoint Output

Paste screenshot/output showing Node Exporter running and metrics exposed on localhost:9100/metrics.



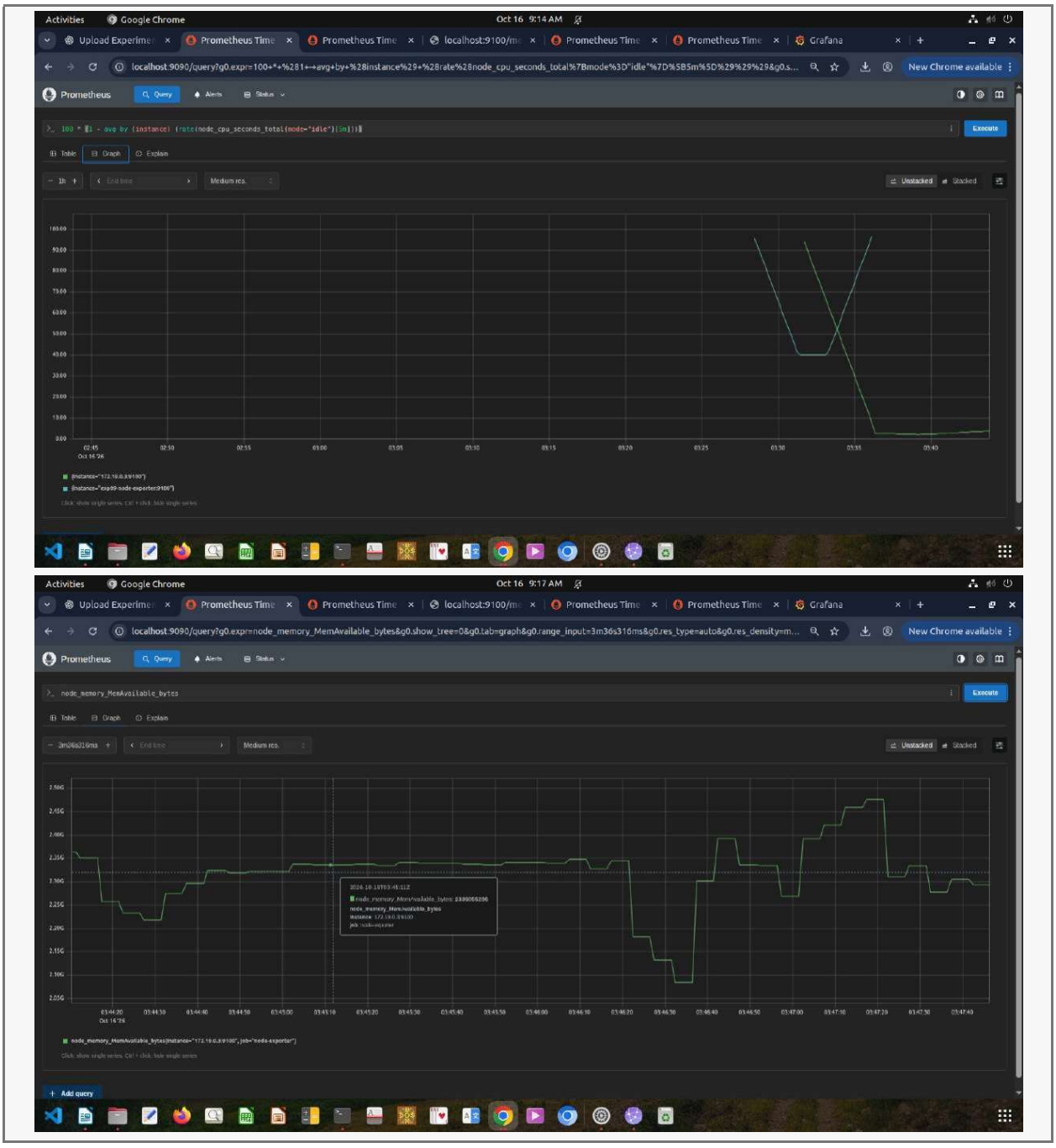
2. Prometheus Configuration and Target Status Output

Paste screenshot/output showing prometheus.yml scrape job and Prometheus target status as UP.



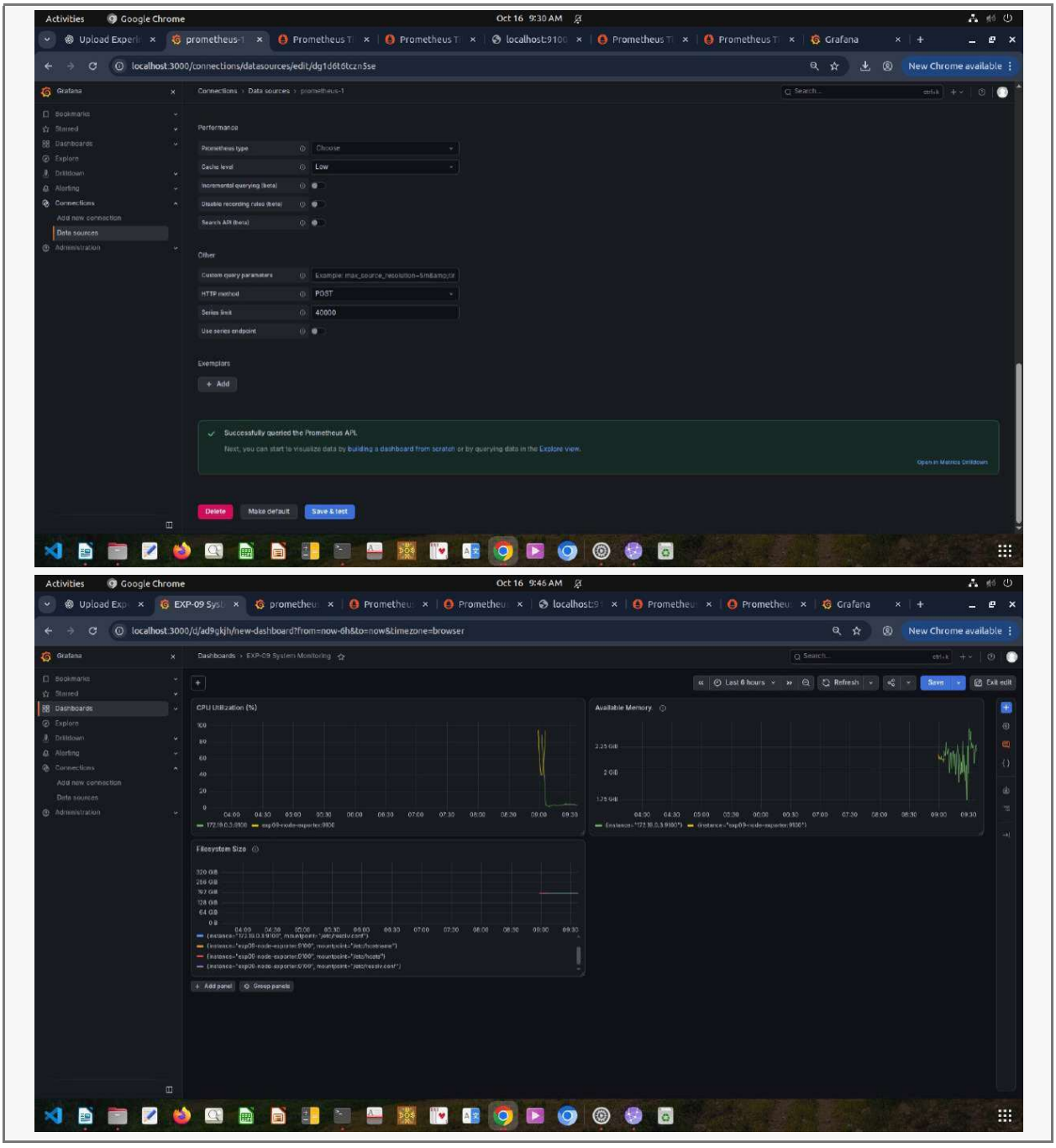
3. PromQL Query Output

Paste screenshot/output showing execution of a PromQL query and the returned metric values or graph.



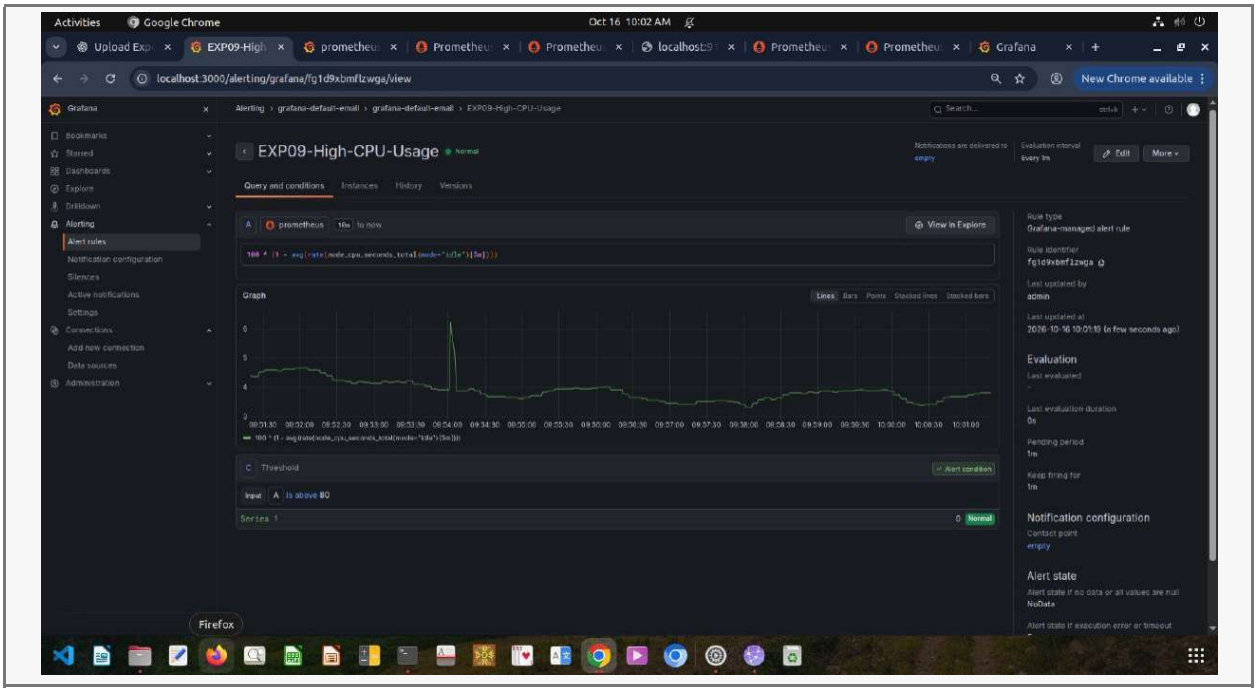
4. Grafana Data Source and Dashboard Output

Paste screenshot/output showing Prometheus added as a Grafana data source and a live dashboard for CPU / memory / disk metrics.



5. Alert Rule / Nagios Monitoring Output

Paste screenshot/output showing an alert rule, threshold condition, notification status or Nagios host/service monitoring status.



Prometheus URL:

http://localhost:9090

Grafana Dashboard URL:

http://localhost:3000 (dashboard: EXP-09 System Monitoring)

GitHub Repository Link / Configuration Folder Link:

Not uploaded to GitHub - configuration is kept on the local machine (/home/apsit/mlflow-monitoring/prometheus.yml).

Final Observation Statement

Node Exporter was run as a Docker container (port 9100) and Prometheus scraped it, showing the target as UP. PromQL queries for CPU utilisation, available memory and filesystem size were executed, Prometheus was added as a Grafana data source, and a dashboard with CPU, memory and filesystem panels was created. A Grafana alert rule, EXP09-High-CPU-Usage, was configured and is in the Normal state.

Expected Output

Prometheus shows the exporter target as UP and serves metrics; Grafana renders a live dashboard of CPU, memory and disk metrics; the configured alert fires when its threshold is crossed.

Conclusion

End-to-end monitoring was implemented by scraping metrics with Prometheus, visualizing them in Grafana and configuring an alert, demonstrating observability of the DevOps stack.

Viva / Review Questions

1. Differentiate pull-based monitoring from push-based monitoring.

2. What is an exporter in Prometheus?

3. What is PromQL used for?

4. What is the difference between monitoring and alerting?

Marks: ____ / 10	Date: _____	Signature of Faculty: _____